WEEK 6 – PYTHON DEVELOPMENT PROJECT REPORT

SHOPPING CART MANAGEMENT SYSTEM

Python Development Internship

Submitted by: Chandana Jakkula

Project Name: Shopping Cart Management System

Internship Domain: Python Development

Assignment: Week 6 – Final Project

1. INTRODUCTION

The Shopping Cart Management System is a console-based Python application developed to manage basic shopping activities.

This project allows users to view available products, search for products, add products to the cart, update quantities, remove items, calculate the total price, apply discounts, and complete checkout.

The application also generates an order reference and saves purchase details in a text file. Users can view their previous bills whenever required.

This project demonstrates the practical application of Python programming concepts, including data structures, functions, file handling, exception handling, standard libraries, and Object-Oriented Programming.

2. PROBLEM STATEMENT

In a shopping process, customers need a simple way to select products, manage quantities, calculate prices, and maintain purchase records.

Manually calculating bills and managing product details can be time-consuming.

The Shopping Cart Management System provides a simple solution by automating product selection, cart management, discount calculation, checkout, and bill generation using Python.

3. OBJECTIVES

1. To develop a shopping cart application using Python.
2. To understand and implement Python fundamentals.
3. To use lists, tuples, sets, and dictionaries in a real-world application.
4. To implement functions and Object-Oriented Programming concepts.
5. To perform file handling operations.
6. To handle errors using exception handling.
7. To use Python standard libraries.
8. To understand Git and GitHub for project management.

4. FEATURES OF THE PROJECT

1. **Display Products:** Displays all available products with their details and prices.
2. **Search Products:** Allows users to search for products by name.
3. **Add to Cart:** Users can add products and specify quantities.
4. **Update Cart:** Allows users to change product quantities.
5. **Remove Products:** Users can remove unwanted products from the cart.
6. **View Cart:** Displays selected products and their quantities.
7. **Calculate Total:** Calculates the total shopping amount.
8. **Apply Discounts:** Supports percentage-based and fixed-amount discounts.
9. **Checkout:** Completes the shopping process and generates an order reference.
10. **Generate Bill:** Saves shopping details in a text file.
11. **View Previous Bills:** Allows users to view previously saved bills.

5. TECHNOLOGIES USED

Technology	Purpose
Python	Backend programming
VS Code	Code editor
Git	Version control
GitHub	Project repository
Text File	Bill storage

Python Libraries Used

- **random:** Used to generate random order references.
- **math:** Used for mathematical calculations.
- **datetime:** Used to store and display date and time.
- **abc:** Used to implement abstraction using abstract classes.

6. PROJECT STRUCTURE

The project is organized into the following structure:

Shopping_Cart_Management_System/

- main.py
- README.md
- shopping_bills.txt
- screenshots/
- Documentation/

File Description:

- **main.py:** Contains the main Python application code.
- **README.md:** Provides project information, features, and instructions.
- **shopping_bills.txt:** Stores generated shopping bills.
- **screenshots/:** Contains screenshots of application execution.
- **Documentation/:** Contains the final project report.

7. IMPLEMENTATION

7.1 Product Management

The application maintains product information such as product ID, name, price, and available stock.

Users can view the available products and search for a particular product.

Explanation:

Product information is maintained using Python data structures. This makes it easier to access product details and manage stock.

Learning Outcome:

Learned how to store, access, and manage product information using Python.

7.2 Shopping Cart Management

Users can add products to the cart, update quantities, remove products, and view selected items.

Explanation:

The shopping cart manages selected products and their quantities. It also checks product availability before adding items.

Learning Outcome:

Understood how to implement real-world shopping cart operations using Python.

7.3 Price Calculation and Discounts

The application calculates the total price of selected products and applies discounts.

Two discount methods are implemented:

1. Percentage Discount
2. Fixed Amount Discount

Explanation:

The system calculates the total amount based on product prices and quantities. It then applies the selected discount to calculate the final payable amount.

Learning Outcome:

Learned how to perform calculations and implement discount logic using Python.

7.4 Checkout and Bill Generation

After selecting products, users can complete the checkout process.

The application generates an order reference and saves the bill details in a text file.

Explanation:

The datetime library is used to record the purchase date and time. File handling is used to save the bill for future reference.

Learning Outcome:

Understood how to generate bills and store purchase information using file handling.

7.5 Previous Bill Management

The application allows users to view previously generated bills.

Explanation:

The saved bill information is read from the text file and displayed to the user.

Learning Outcome:

Learned how to read and display stored information from files.

8. PYTHON CONCEPTS USED

8.1 Variables and Data Types

Variables are used to store product names, prices, quantities, totals, and other information.

Python data types such as integers, floats, and strings are used in the project.

Learning Outcome: Understood how to store and manage different types of data.

8.2 Input and Output

The input() function is used to receive user choices and product quantities.

The print() function is used to display product information, cart details, and bills.

Learning Outcome: Learned how to interact with users through a console application.

8.3 Operators

Arithmetic operators are used for price calculations.

Examples:

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Modulus (%)

Learning Outcome: Understood how operators are used in real-world calculations.

8.4 Conditional Statements

if, elif, and else statements are used to check user choices, product availability, and shopping conditions.

Learning Outcome: Learned how to implement decision-making in Python.

8.5 Loops

Loops are used to display menus, process user choices, and repeat shopping operations.

Learning Outcome: Understood how loops help automate repeated operations.

8.6 Functions

Functions are used to divide the application into smaller reusable operations.

Examples include displaying products, searching products, adding items, calculating totals, and checkout.

Learning Outcome: Learned how to write reusable and organized code using functions.

8.7 Data Structures

The project uses the following Python data structures:

List: Used to manage collections of products and cart information.

Tuple: Used to maintain fixed information such as product categories.

Set: Used to maintain unique values and avoid duplicate entries where required.

Dictionary: Used to store product details and related information using key-value pairs.

Learning Outcome: Understood how different Python data structures are used in practical applications.

8.8 File Handling

File handling is used to save shopping bills in shopping_bills.txt.

The application writes purchase information into the file and reads previous bills when requested.

Learning Outcome: Learned how to create, read, and write files using Python.

8.9 Exception Handling

try and except blocks are used to handle errors such as invalid user input and incorrect values.

This prevents the application from stopping unexpectedly when certain errors occur.

Learning Outcome: Understood how to handle runtime errors and improve application reliability.

8.10 Standard Libraries

The project uses Python's built-in libraries:

- random – Generates order references.
- math – Supports mathematical calculations.
- datetime – Handles date and time.

- abc – Supports abstract classes.

Learning Outcome: Learned how to use Python standard libraries in application development.

9. OBJECT-ORIENTED PROGRAMMING CONCEPTS

9.1 Classes and Objects

Classes are used to organize the application into different components.

The project includes classes such as Product, ShoppingCart, and ShoppingSystem.

Objects are created from these classes to perform shopping operations.

Learning Outcome: Understood how classes and objects represent real-world entities.

9.2 Constructors

The **init**() constructor is used to initialize object attributes when an object is created.

Learning Outcome: Learned how to initialize object data using constructors.

9.3 Inheritance

Inheritance is implemented through the discount classes.

PercentageDiscount and FixedDiscount inherit common functionality from the Discount base class.

Learning Outcome: Understood how inheritance supports code reusability.

9.4 Polymorphism

Polymorphism is used through different discount classes that implement their own discount calculation behavior.

Learning Outcome: Learned how different classes can provide different implementations of a common method.

9.5 Encapsulation

Encapsulation is used to organize and manage data and methods within classes.

The shopping cart manages its internal cart information through its methods.

Learning Outcome: Understood how encapsulation helps organize and manage application data.

9.6 Abstraction

Abstraction is implemented using Python's abc module and an abstract Discount class.

The abstract class defines a common discount interface, while child classes provide their own implementations.

Learning Outcome: Learned how abstraction hides implementation details and defines a common structure.

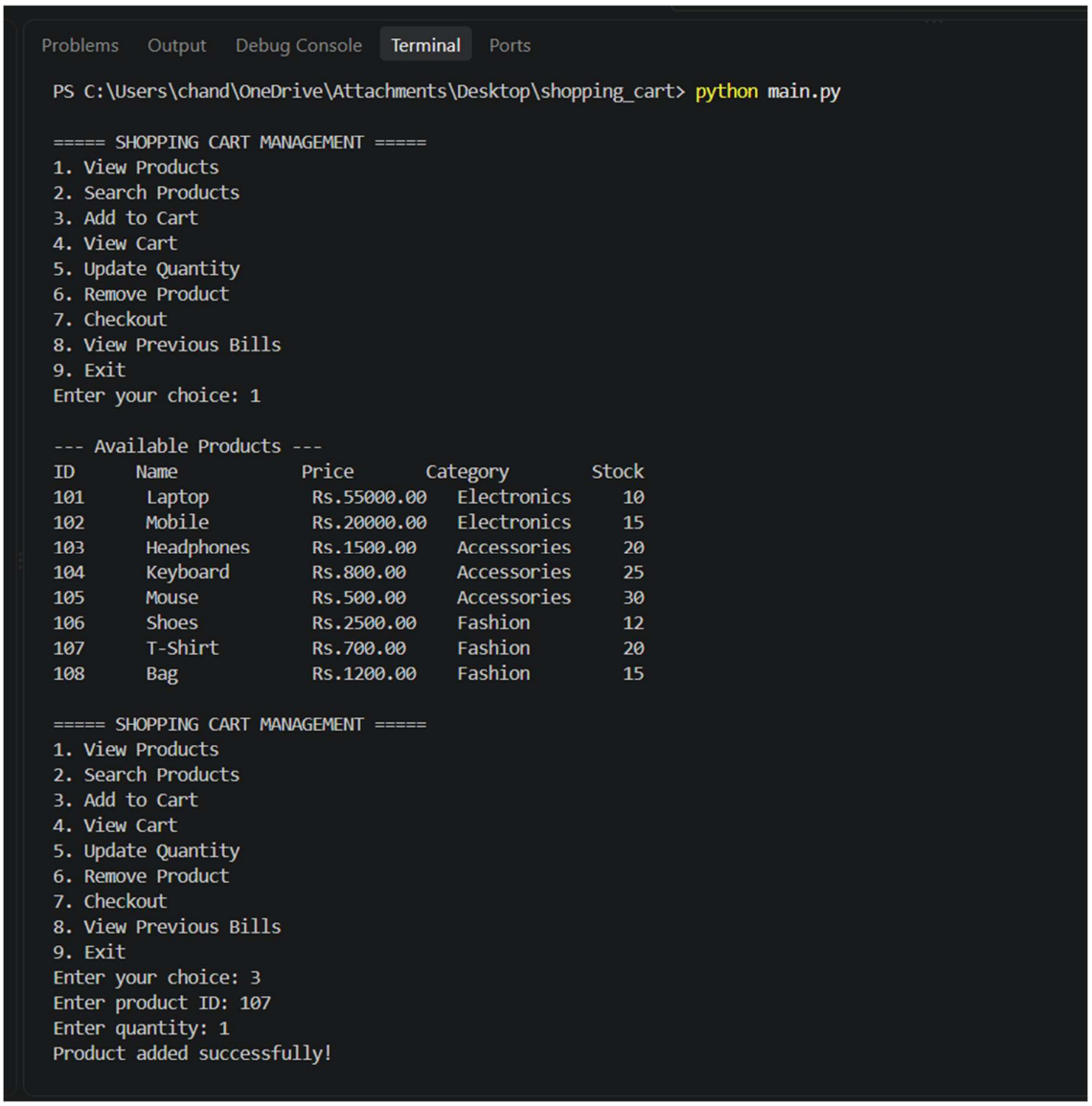
10. WORKFLOW OF THE PROJECT

1. Start the Python application.
2. Display the main shopping menu.
3. Display available products.
4. Search and select products.
5. Add products to the cart.
6. Update or remove cart items if required.
7. Display the cart and calculate the total.
8. Apply an available discount.
9. Complete checkout.
10. Generate an order reference.
11. Save the bill in a text file.
12. View previous bills whenever required.

11. SAMPLE OUTPUTS AND SCREENSHOTS

The following screenshots show the execution of the Shopping Cart Management System.

Screenshot 1: Main Menu and Product Display



```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\chand\OneDrive\Attachments\Desktop\shopping_cart> python main.py

===== SHOPPING CART MANAGEMENT =====
1. View Products
2. Search Products
3. Add to Cart
4. View Cart
5. Update Quantity
6. Remove Product
7. Checkout
8. View Previous Bills
9. Exit
Enter your choice: 1

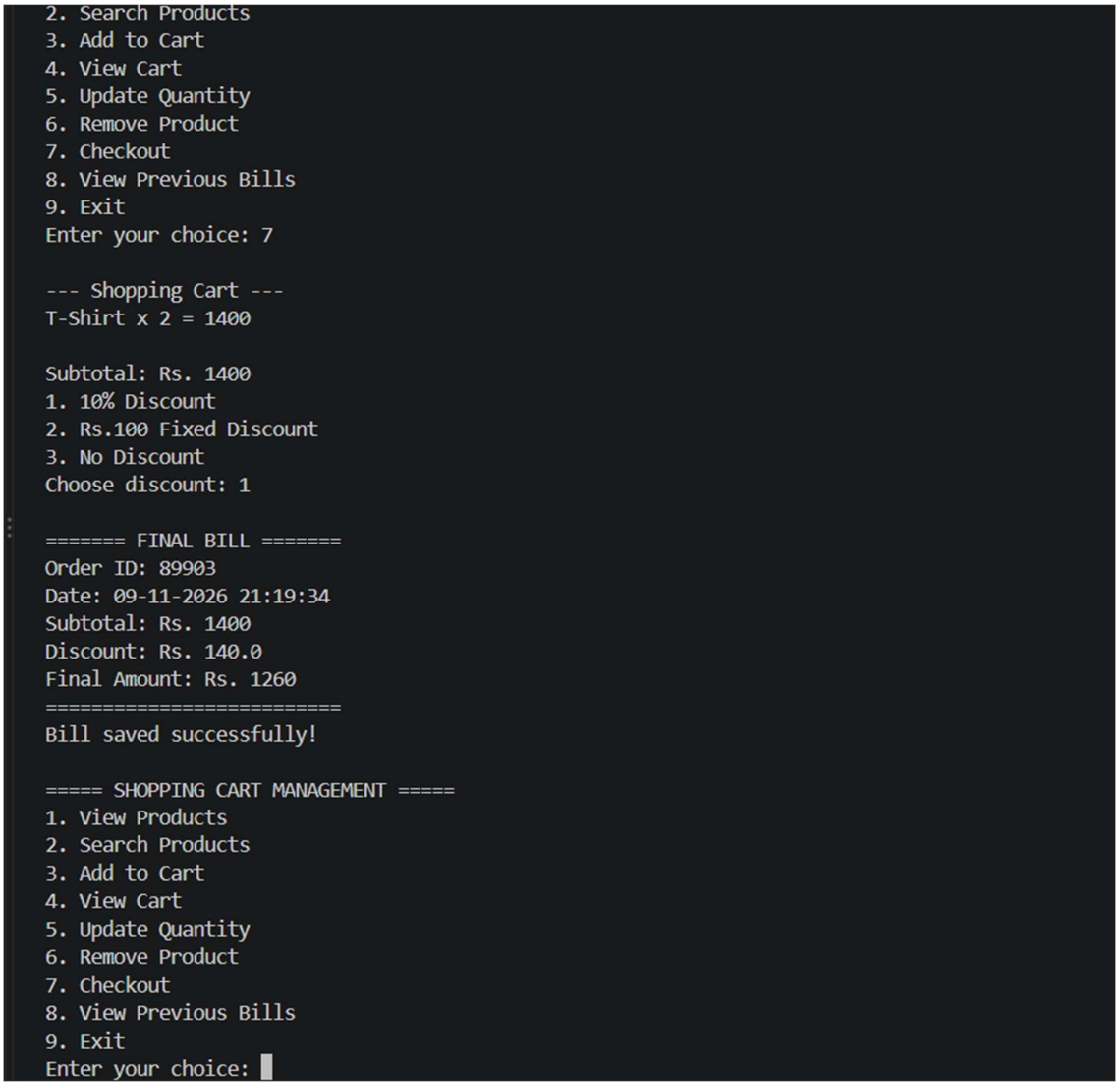
--- Available Products ---
ID      Name            Price       Category        Stock
101      Laptop          Rs.55000.00   Electronics     10
102      Mobile          Rs.20000.00   Electronics     15
103      Headphones      Rs.1500.00    Accessories     20
104      Keyboard        Rs.800.00     Accessories     25
105      Mouse           Rs.500.00     Accessories     30
106      Shoes           Rs.2500.00    Fashion         12
107      T-Shirt         Rs.700.00     Fashion         20
108      Bag             Rs.1200.00    Fashion         15

===== SHOPPING CART MANAGEMENT =====
1. View Products
2. Search Products
3. Add to Cart
4. View Cart
5. Update Quantity
6. Remove Product
7. Checkout
8. View Previous Bills
9. Exit
Enter your choice: 3
Enter product ID: 107
Enter quantity: 1
Product added successfully!
```

Screenshot 2: Adding Products to Cart



Screenshot 3: Cart Details and Discount Calculation



Screenshot 4: Checkout and Bill Generation



12. TESTING

The application was tested using different shopping operations.

Test Case	Expected Result
Display products	Products should be displayed
Search product	Matching product should be displayed
Add product	Product should be added to cart
Update quantity	Quantity should be updated
Remove product	Product should be removed
Calculate total	Correct total should be calculated
Apply discount	Discount should be applied
Checkout	Order reference should be generated
Save bill	Bill should be stored in a text file
View previous bills	Saved bills should be displayed
Invalid input	Error should be handled properly

13. CHALLENGES FACED

1. Managing product details and cart information using Python data structures.
2. Calculating the total amount and applying discounts correctly.
3. Implementing different Object-Oriented Programming concepts.
4. Handling invalid user inputs and runtime errors.
5. Saving and retrieving shopping bills using file handling.
6. Organizing the project files and uploading the project to GitHub.

These challenges helped improve my Python programming and problem-solving skills.

14. GIT AND GITHUB IMPLEMENTATION

Git was used for version control, and GitHub was used to store and manage the project repository.

The following Git commands were used:

- git init – Initializes a local Git repository.
- git status – Checks the current repository status.
- git add . – Adds project files to the staging area.
- git commit -m "message" – Saves changes in the repository.
- git push – Uploads project files to GitHub.

The final project was uploaded to GitHub for storage and sharing.

GitHub Repository Link:

https://github.com/jakkula-chandana/Shopping_Cart_Management_System

15. FUTURE IMPROVEMENTS

The following features can be added in the future:

1. Develop a graphical user interface.
2. Integrate a database such as MySQL.
3. Add user registration and login.
4. Add online payment functionality.
5. Maintain permanent product stock updates.
6. Generate downloadable PDF bills.
7. Develop a web-based shopping application using Django or Flask.

16. LEARNING OUTCOMES

Through this project, I learned:

1. How to develop a real-world application using Python.
2. How to use lists, tuples, sets, and dictionaries.
3. How to implement functions and modular programming.
4. How to perform file handling operations.
5. How to handle errors using exception handling.

6. How to use Python standard libraries.
7. How to implement all major Object-Oriented Programming concepts.
8. How to manage source code using Git and GitHub.
9. How to organize and document a complete Python project.
10. How to apply programming concepts to solve practical problems.

17. CONCLUSION

The Shopping Cart Management System was successfully developed using Python.

This project helped me understand and apply Python fundamentals, data structures, file handling, exception handling, standard libraries, and Object-Oriented Programming concepts in a practical application.

I also gained experience in using Git and GitHub for project management and documentation.

Overall, this project improved my Python programming knowledge, logical thinking, and practical development skills.

THANK YOU